
PAYOUT API VERSION 2

Version 2.2.1 – May 2019

Copyright © WorldRemit 2018-2019

Paul Stancer

This document explains our new PayOut API, which you can use to integrate your applications and systems directly with the WorldRemit pay-out infrastructure. This API has been completely rewritten from the earlier version and has gained significant new functionality.

We hope this new API will reduce the integration work needed to connect to our systems.

Please note: the functionality that is available to you will depend on the types of products and services that you provide; consequently, some of the methods in this API may be unavailable. If you require a feature that is not enabled, please speak to the WorldRemit Operations team or your WorldRemit Business Development representative.

We would also encourage your feedback on this API and welcome your comments, and suggestions for new features.

CONTENTS

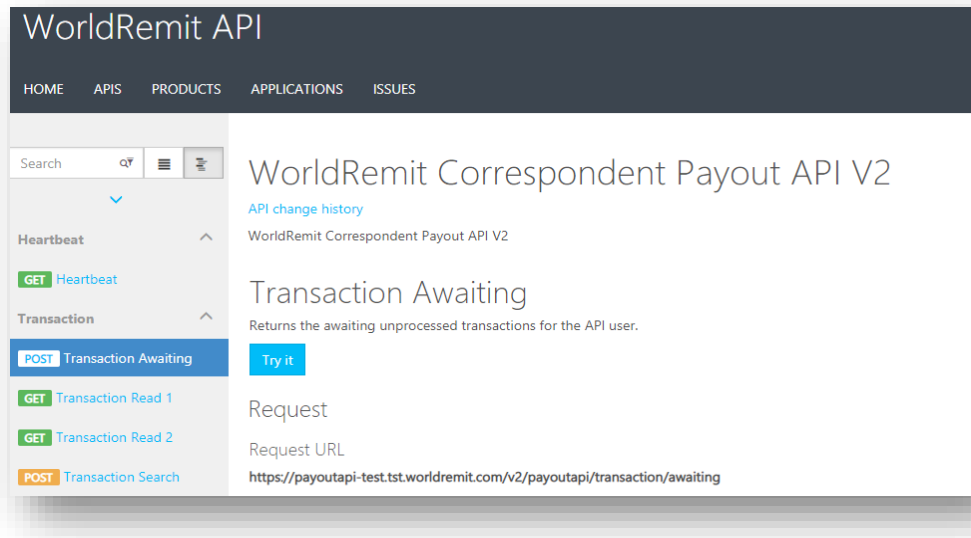
PayOut API Version 2	1
Document History	3
What's new?	4
Using the Sandbox Environment.....	5
API Developer Portal.....	5
Registering with the Developer Portal.....	5
Receiving Credentials for the Sandbox API	7
Sending a test API request	8
Securing the API	10
Digital signature of outgoing transaction data.	10
Audit Functionality.....	10
Rate Limiting and 429 errors.....	10
Penetration Testing and Port Scanning	10
Connector Payout Integration Library	11
Communication Diagram	11
Connector Payout Library Usage.....	12
Payout Connector License	13

PayOut API v2.....	14
Supported Products	14
Example Transaction Flow	15
API Response Codes	16
Paths	16
Heartbeat.....	16
Transaction.....	16
Transaction State	17
Transaction Comment.....	20
Transaction Batch	22
Transaction Batch Lot	23
Transaction Cancellation.....	24
Transaction Change Beneficiary.....	24
Transaction Status.....	25
Transaction Notepad.....	26
Validation	27
Webhooks	27
User Capabilities	30
User Notepad	30
Correspondent Notepad	31
Sandbox API Overview	32
Paths	32
Transaction Create	32
Transaction.....	33
Transaction Cancel	34
Transaction Beneficiary Change.....	34
Transaction State	35
API examples.....	36
Manual Cash pick-up using the API.....	36
Hello	Error! Bookmark not defined.
Cash Pick-up with SMS verification code (S2FA).....	36
Synchronous mobile money transfer.....	Error! Bookmark not defined.
Asynchronous bank transfer	37

DOCUMENT HISTORY

Version	Date	Authors	Description
2.2.1	May 2019	Yordan Byalkov	Added sequence diagram for an Async Integration process
2.2.0	January 2019	Paul Stancer	Incorporated feedback from Correspondent partners and added new functionality for failure response codes and additional API features.
2.1.4	October 2018	Paul Stancer	Added additional Sandbox API method to create test transactions.
2.1.3	September 2018	Paul Stancer	- Added addition Notepad features to the API. - Added User Capabilities feature. - Fixed registration errors for Webhooks. - Expanded Tech examples and flows, with sequence diagrams.
2.1.2	September 2018	Paul Stancer, Mikko Tormanen, Guy Steel	- Expanded on tech examples. - API error codes extended - Example JSON files provided
2.1.1	September 2018	Paul Stancer, Guy Steel	- Expanded integration examples - Completed Sandbox API section - Added APIM Permalinks - Swagger definition extended, and additional error codes added.
2.1	August 2018	Paul Stancer, Guy Steel	- Updated API definitions. - Expanded API portal sections and provided links to the online portal. - Removed detailed API description and ported these to the online API portal. - Added references to the NuGet package. - Added a detailed description of the Sandbox service
2.0	July 2018	Paul Stancer	Initial draft release for review.

WHAT'S NEW?



The PayOut API Version 2 is a REST-based implementation that uses JSON for the transport mechanism. It has several new key features that provides an improved service to our correspondent partners. These features are:

- Instant Notifications for pending transactions and actions using remote Webhook URL call-backs.
- Enhanced transaction search capabilities with pagination support.
- NuGet connector library package for simplifying accessing to the PayOutAPI v2 when using Microsoft .NET based programming languages.
- Support for Cash pick-up two-factor authentication using SMS, in countries where this is a legal requirement.
- Notepad functionality – the ability to store reference and associated information against a transaction, correspondent and for the calling user.
- Dynamic Account Validation support to reduce invalid transaction requests to a sender.
- Cancellation and Change Beneficiary requests are now supported by the API.
- Direct Status Requests can be retrieved through the API.
- Enhanced Security protocols
 - JWT bearer token-based authentication;
 - Mutual TLS certificate support;
 - IPv4 and IPv6 range restriction supported when required.

Please note: the detailed API and model definitions are in our online portal. This document only contains a high-level description of the capabilities and descriptions on how to use the API. For detailed definitions you will need to register with the online API portal.

USING THE SANDBOX ENVIRONMENT

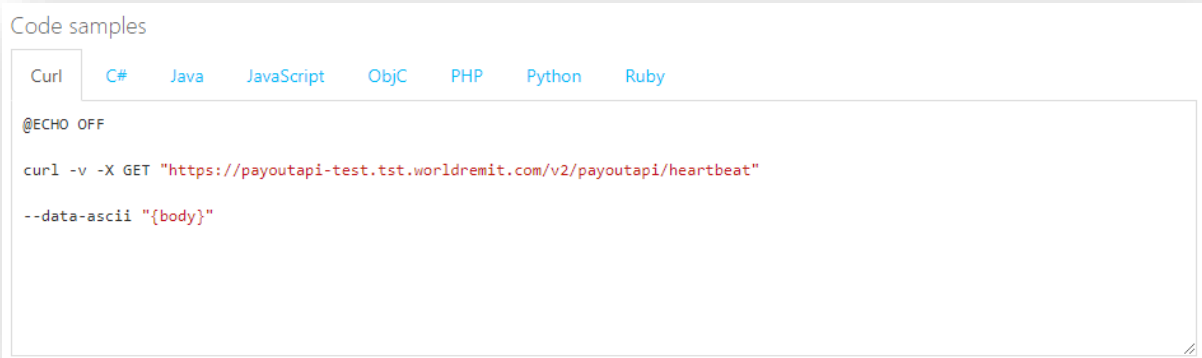
The Sandbox environment is a new system that is an isolated stand-alone test simulator of our new pay-out systems. You can use this environment for both your development work, and for your integration testing. This environment will always match our current production environment.

API Developer Portal

The portal is built using Microsoft's [Azure API Management service](#). It is an enterprise grade API service built upon a scalable and high-capacity service.

You can access the API for testing purposes by means of the portal. This portal details all the API calls that are available, as well as the schema definitions of the JSON-based models that we use to transfer information, and a detailed explanation of any error codes returned.

The portal also provides simple code examples in a variety of languages, documenting how you can call the API.

A screenshot of the 'Code samples' section in the API Developer Portal. It features a tabbed interface with 'Curl' selected. Below the tabs, the text '@ECHO OFF' is visible. The main content area displays a curl command: 'curl -v -X GET "https://payoutapi-test.tst.worldremit.com/v2/payoutapi/heartbeat"' followed by '--data-ascii "{body}"' on a new line. The code is color-coded: 'curl' is blue, '-v' is red, '-X GET' is red, the URL is red, and the data body is red.

```
Code samples

Curl  C#  Java  JavaScript  ObjC  PHP  Python  Ruby

@ECHO OFF

curl -v -X GET "https://payoutapi-test.tst.worldremit.com/v2/payoutapi/heartbeat"

--data-ascii "{body}"
```

However, if you are using a Microsoft technology to connect to our API, then we recommend that you use our [PayOut API Connector NuGet library](#).

Registering with the Developer Portal

You will need to register to access our developer portal. This enables you to see the API documentation, our swagger definitions and the ability to make test calls against the API.

The first step is to create an account using the following link:

- <https://corint-apim-tst-wr.portal.azure-api.net/signup>

You will see the following page:

Sign up

Already a member? [Sign in now](#)

Create a new API Management account

Email

job.blogs@gmail.com

Password

.....

Very Strong

Confirm password

.....

First name

Joe

Last name

Blogs

Enter the characters you see

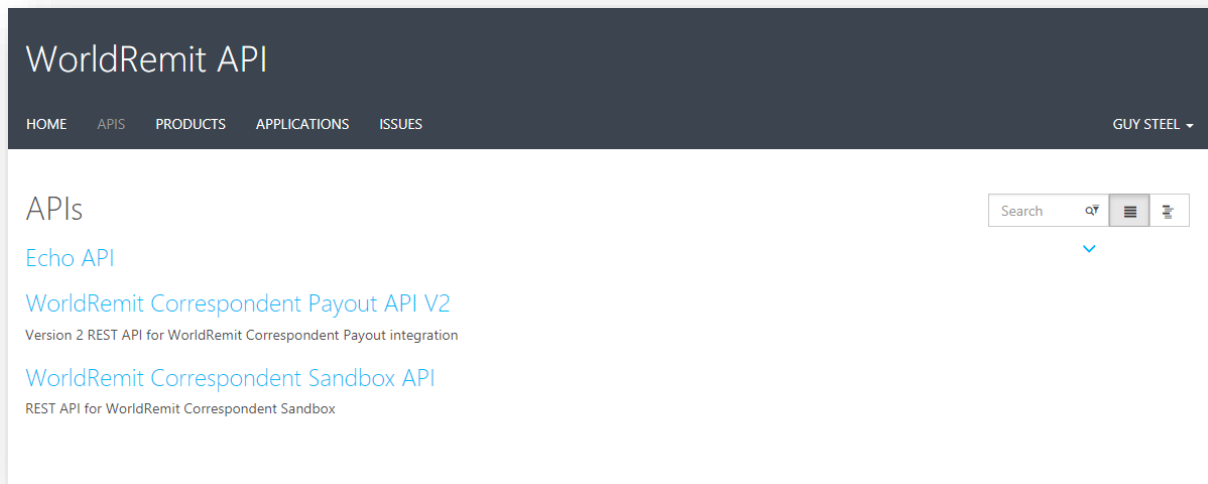
[New](#) | [Audio](#)



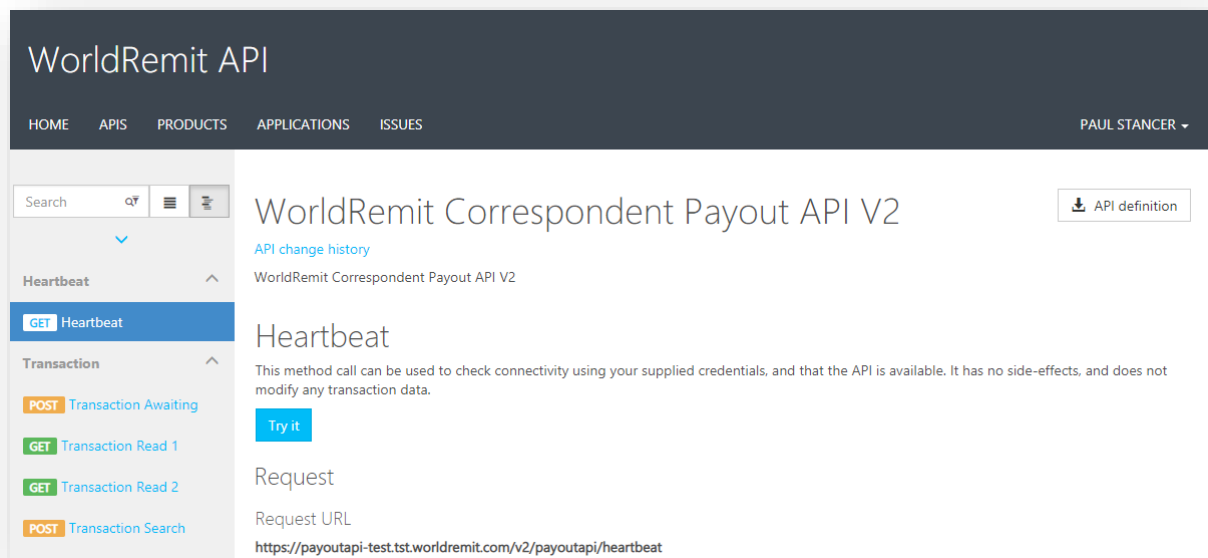
VLKS

[Sign up](#)

Open an account using your email address and a new password. You will be emailed a link to verify your credentials. Once your account has been created and verified (via an email activation link) you can sign in and click on the main *APIs* menu (as below):



If you click on the PayOut API link, you will see the following page where all the API calls are on the left-hand bar. You can select an individual API method by clicking on the link.



You can group the method calls on the left either alphabetically or by function group by use of the toggle switch next to the search bar.

Receiving Credentials for the Sandbox API

You will need two sets of credentials to access the Sandbox service.

- **PayOut API JWT token** – this token will allow you to make API requests against the main interface, to retrieve pending transactions and other requests for your account.
- **Sandbox API JWT token** – the Sandbox API is still in development and has not been fully released yet. This API allows you to create test transactions and upload them in to the Sandbox environment, so that you can then retrieve them through the PayOut API. This token allows for you to create automated integration tests to run against our system to validate any code changes on either system.

The WorldRemit operations team will send you the necessary credentials as part of your onboarding process. If you are having trouble accessing the API, please email correspondent.onboarding@worldremit.com.

Sending a test API request

Clicking on the **WorldRemit Correspondent PayOut API V2** link displays all the operations associated with the PayOut API. Clicking on an operation displays the associated documentation and provides an option to **Try it** which allows a call to be made against the sandbox environment. As an example, the following shows a call to the **Heartbeat** operation:

WorldRemit Correspondent Payout API V2

Heartbeat

This method call can be used to check connectivity using your supplied credentials, and that the API is available. It has no side-effects, and does not modify any transaction data.

Query parameters

[+ Add parameter](#)

Headers

Authorization

Bearer eyJhbGciOiJIUzI1

[✕ Remove header](#)

[+ Add header](#)

Request URL

https://payoutapi-test.tst.worldremit.com/v2/payoutapi/heartbeat

HTTP request

GET https://payoutapi-test.tst.worldremit.com/v2/payoutapi/heartbeat HTTP/1.1
Host: payoutapi-test.tst.worldremit.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG91IiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6y3J_adQssw5c

Send

Please note that the Authorization header must be added before the request is sent otherwise the API will respond with a **401 Unauthorized** response. The PayOut API JWT token must be preceded with the word **Bearer** as shown above. The subsequent response from the API call is shown underneath the request:

Response status

200 OK

Response latency

73 ms

Response content

```
Transfer-Encoding: chunked
Request-Context: appId=cid-v1:3b74f9e2-4f7f-4791-8a62-b11fc1693061
Date: Wed, 15 Aug 2018 17:40:20 GMT
Set-Cookie: ARRAffinity=b1ad293de292860eb094b69fa793aad6a45ce9d33f974ccb12d9d3c808683ece;Path=/;HttpOnly;Domain=corint-microser
vice-webapp-tst-wr.azurewebsites.net
X-Powered-By: ASP.NET
Content-Type: text/plain; charset=utf-8
```

```
D05BD2CEF6854775A7BA69905D1EA482-2018-08-15T17:40:20.6126333Z
```

In this case the heartbeat responds with a unique identifier and a timestamp. This operation can be used to verify that the service is up and running.

SECURING THE API

The API security is primarily based around JWT tokens issued by the WorldRemit operations team. Each token is specific to an environment and an API user, and is not be transferable, i.e. a sandbox token will not work on either the UAT or production API environments.

In addition, you may be provided with a client-side TLS certificate to sign your requests to the PayOutAPI v2. This certificate will be specific to you and the environment and will be supplied by the WorldRemit Operations team.

All requests to the PayOut API v2 must be over HTTPS and secured using TLS 1.2 or later.

In addition, you may request IP address restrictions to be applied to your account. Calls to the API using your credentials from IP addresses other than the ones that you have specified, will be rejected.

Digital signature of outgoing transaction data.

To ensure that the transaction information has not be altered within our system, we apply a signed digital signature to specific fields within the transaction.

This is exposed as the 'Signature' field on the Transaction object. The field is a JWT token that contains two SHA-512 hashes of both the minor fields and the entire transaction object. The token that contains these hashes is signed using a public/private key pair.

You will be able to validate the token using the public key, which will be provided by our Operations team on request.

Currently this functionality is only available on our development servers, but will be exposed to the Sandbox and our production systems in the next release.

Audit Functionality

Each request to our API and the subsequent response are stored within our Audit system. We use this to monitor usage and identify timing trends to trigger automated alerts to our Operation team for slowdowns or potential issues with your connection.

Rate Limiting and 429 errors

Please be aware that we impose limits on your usage of our API. Volume and/or stress tests against our API should be agreed and scheduled in advance before any tests are run. If this is not done, you may trigger our rate limit restrictions and you will receive 429 HTTP errors to your API calls, until such time these limits are reset by the WorldRemit Operations Team.

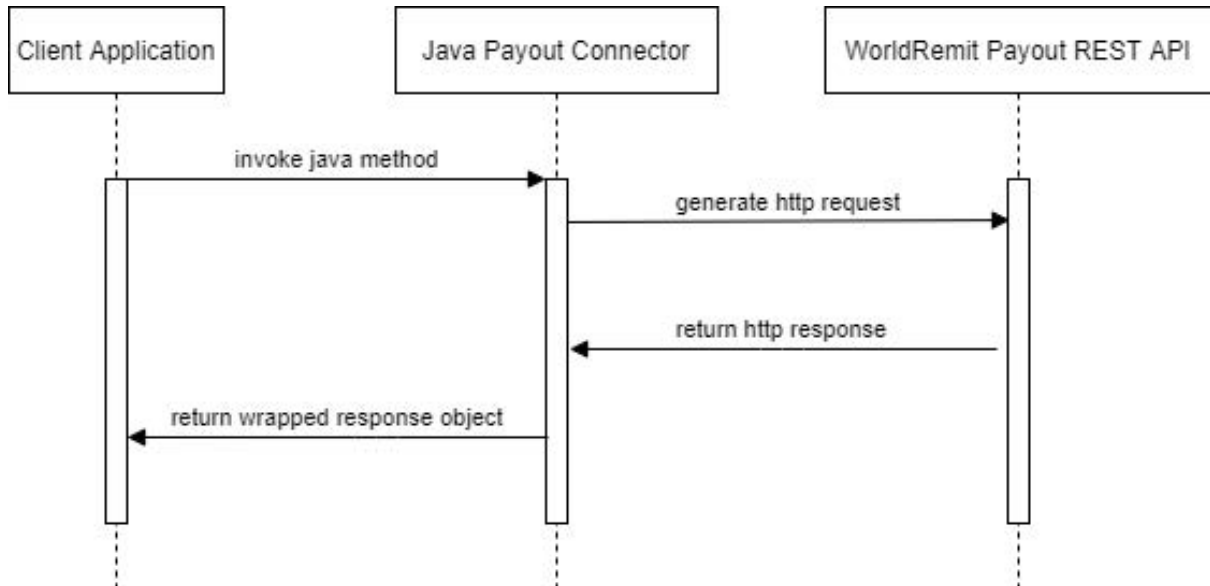
Penetration Testing and Port Scanning

Please note that penetration testing and/or port scanning against our endpoints should not be run without our express approval. This may result in your IP address ranges being automatically blocked by our API monitoring software.

CONNECTOR PAYOUT INTEGRATION LIBRARY

This is a library for simplest integration with WorldRemit payout API based on REST client and uses JWT authorization. The library build on Java.

Sequence Diagram



Connector Payout Library Usage

1. Add library archive(payout-api-connector-0.0.1.jar) to your project as a build dependency.
2. Create com.worldremit.connector.client.HttpEndpoint object:
HttpEndpoint httpEndpoint = newHttpEndpoint({wr_rest_api_url});
3. Add authorization header:
httpEndpoint.addHeader(new JwtHeader({jwt_header}));
4. Create general connector API object, once created paymentApi object can be used to make calls to the API:
PaymentApi paymentApi = PaymentApi.newClient(httpEndpoint).build;
OR
PaymentApi.ClientBuilder builder = PaymentApi.newClient(httpEndpoint);
builder.setConnectionLiveMillis({milis});
builder.setEvictExpiredConnections({true});
....
PaymentApi paymentApi = builder.build();
5. Get specific request provider (implementation) via PaymentApi object:
BatchApi batchApi = paymentApi.createApiInstance({BatchApi.class});
6. Invoke WR API using java method:

ApiResponse<Batch> response = batchApi.create(new Batch());

NOTES:

- all specific providers are located in package: com.worldremit.connector.api;
- all variables in above instruction are marked with {};
- the following library uses Apache HttpClient. Client is configured by default, for client customization please use PaymentApi.ClientBuilder class during creating general connector object;

Payout Connector License

Copyright © 2018 WorldRemit. Subject to you having entered into a separate commercial agreement with WorldRemit for the use by you of WorldRemit services (the “Agreement”), WorldRemit hereby grants any user of this library (“you”) a revocable, non-transferable, non-exclusive, limited license to use this library for the sole purposes of integrating your system with the **WorldRemit Correspondent PayOut APIs**. Please refer to the terms and conditions set out in your Agreement with WorldRemit, which apply to your use of this package. This package shall constitute part of WorldRemit’s intellectual property rights for the purposes of the Agreement.

For the avoidance of doubt, you may not use any such information provided by us or obtained by you to create any software whose expression is substantially similar to that of this library nor use such information in any manner which would be restricted by any copyright subsisting in it. The above copyright notice and this permission notice shall be included in all copies or substantial portions of this library.

THIS LIBRARY IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

PAYOUT API V2

This section contains the definition for the new API.

Each User JWT token is mapped to one of more 'Correspondent Ids'. A Correspondent Id is a GUID and is assigned to a transaction based on how your account has been set up the WorldRemit Operations team. Each Correspondent Id will be mapped to your account and potentially further refined in to a specific product type, i.e. Cash pickup, Mobile money transfer, Bank transfer etc.

You will be provided with a list of our internal correspondent Ids by our Operations team. These will be the same across all our environments, including production. Your token will be able to manage transaction for the Correspondent Ids assigned to you.

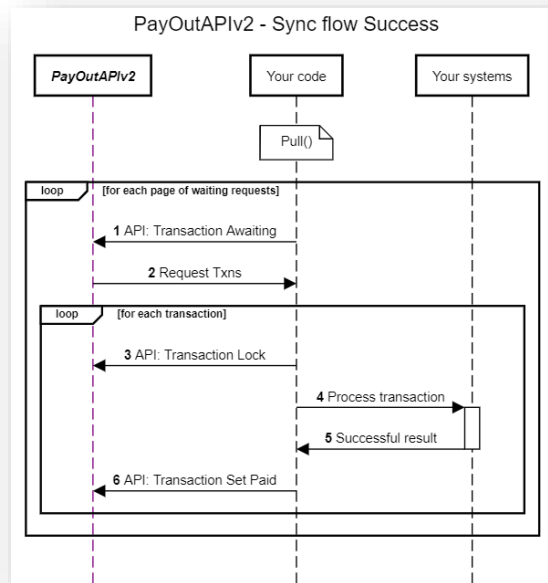
Supported Products

The new API supports as a minimum, the following pay-out services:

- **Cash pickup** - where the recipient receives funds as physical cash at a pay-out location in the receive market.
- **Bank Transfer** - where the recipient receives funds in their bank account in the receive market.
- **Mobile Money** - where the recipient receives funds in their mobile money wallet in the receive market.
- **Card Payments** - where the recipient receives funds on a designated prepaid, debit or credit card.
- **Door to Door** - where the recipient receives funds as physical cash at a chosen residential location.
- **Airtime Top-up** - where the recipient receives direct airtime credit.

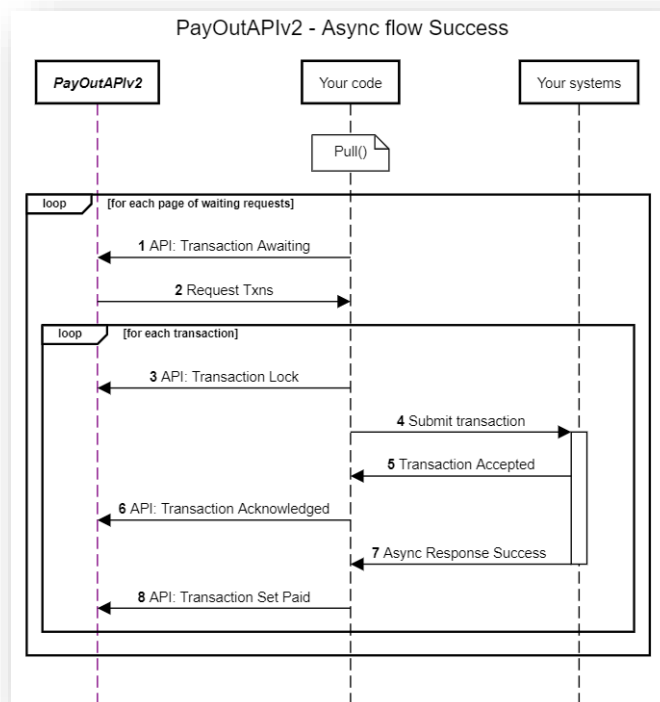
Example Transaction Flow

You should make use of the Awaiting function to retrieve new Transactions, Cancellations, and Change Beneficiary requests.



The standard flow to process awaiting transactions from the PayOut API is as follows.

The process flow for an Asynchronous transaction is as follows.



Any specific use-case examples can be found in the **API examples** section of this document.

API Response Codes

Each of the API request has a set of HTTP response codes associated with them, in addition these

Responses

200 OK

The transaction has been marked as paid.

500 Internal Server Error

There has been an internal exception. Please inform the WorldRemit operations team.

400 Bad Request

Default: The request is badly formed or is missing mandatory data. Please consult API documentation for further details.

- 901 Invalid request passed.
- 905 Invalid identification type
- 923 Invalid beneficiary identification number
- 700 Invalid transaction
- 701 Cannot retrieve transaction state history.

responses will contain a more detailed set of error codes in the HTTP Status message. The codes associated with a specific request can be viewed through the API portal.

Paths

This section contains the PayOutAPI v2 method calls. These calls are grouped around common functionality below.

In addition, each of the API method call documented contain the direct link to the API portal definition, which has the complete definition. You will need to have registered with the portal to view these links (see **Registering with the Developer Portal** for details).

Heartbeat

The heartbeat functionality can be used to confirm that you can connect to our API with your supplied credentials.

GET /heartbeat

Summary

This method call can be used to check connectivity using your supplied credentials and confirm that the API is accessible from your network. It has no side-effects and does not modify any transactional data on the service.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Heartbeat>

Transaction

These functions are designed for the simple retrieval of awaiting transaction, and for the simple processing of the response.

POST /transaction/awaiting

Summary

Returns the awaiting unprocessed transactions for the API user.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Awaiting>

GET /transaction/read

Summary

Reads the full transaction by its unique id or by WorldRemit references:

- **By transaction Id** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Read-1>
- **By transaction reference** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Read-2>

POST /transaction/search

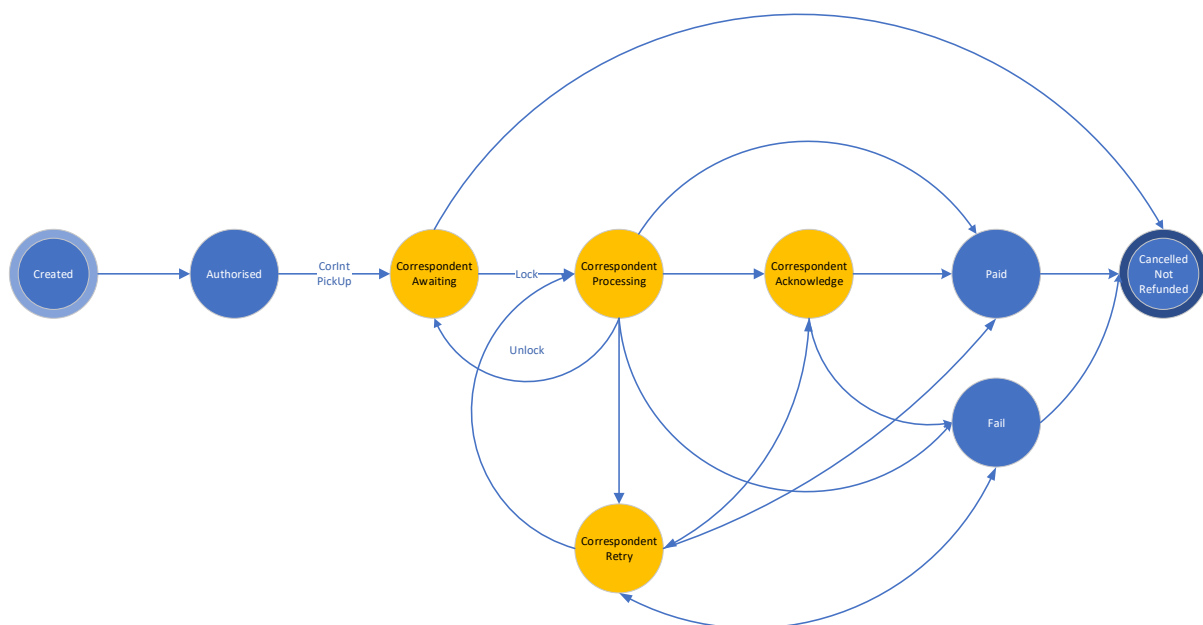
Summary

Searches for the specific transactions based on the parameters supplied in the search request.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Search>

Transaction State

The API supports multiple method calls that allow you to report the current state of a transaction that you are processing back to WorldRemit. The diagram below shows the overall states that a transaction can move between when being processed through the PayOut API.



The actual flow will depend on both the transaction product type (i.e. cash/mobile-money/bank etc), and on whether you process payments asynchronously or synchronously through your system.

The WorldRemit operations team will agree with you the appropriate workflow when your account is set up.

- **Authorized** – the transaction has been authorized for payment and will now be picked up and the correspondent will be notified of the incoming transaction.
- **Correspondent Awaiting** – the transaction can be read from the API from the Awaiting method call.
- **Correspondent Processing** – the correspondent has read the transaction and/or marked the transaction as locked for payment (this depends on the type of product and workflow assigned to your account). We are awaiting a response to the transaction.
- **Correspondent Acknowledge** – this state depends on whether you process transactions asynchronously. This workflow will be assigned to you if there is a delay between reading a transaction, specifically when you are sending the transaction to a remote third-party system for processing.
- **Paid** – the transaction has been marked as paid.
- **Failed** – the transaction has been marked as failed.
- **Retry** – the transaction is delayed, and transmission will be retried later.
- **Cancelled (Not Refunded)** – the transaction has been marked as cancelled. There are additional steps to a transaction flow that will be picked up by our customer service department to ensure that the sender is credited the money that they have sent.

PUT /transaction/set/paid

Summary

Marks the transaction as paid. This is a final transaction state. There are several parameters that you will need to return in this request body for your paid request to be accepted. The

Request body

text/plain application/json-patch+json application/json text/json application/*+json

Sample Schema

```
{
  "validationCode": "string",
  "officeId": 0,
  "beneficiaryIdExpiryDate": "string",
  "beneficiaryIdNumber": "string",
  "beneficiaryIdType": "string",
  "beneficiaryIdPlaceOfIssue": "string",
  "responseCode": "string",
  "responseMessage": "string",
  "comment": "string",
  "properties": {}
}
```

list of mandatory parameters will depend on how your account has been configured.

For example, in certain countries you will need to return a numeric validation code when confirming a cash pick up. This number should be entered in to the 'validationCode' property. If the correct code is not provided this request will fail.

The full list of parameters and their values can be found on the API portal (see below).

- **By transaction Id** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Paid-1>
- **By transaction reference** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Paid-2>

PUT /transaction/set/failed

Summary

Marks a transaction as failed. This operation may not be available to all users.

- **By transaction Id** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Failed-1>
- **By transaction reference** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Failed-2>

PUT /transaction/set/locked

Summary

Marks a transaction as locked. This removes it from the awaiting queue.

- **By transaction Id** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Locked-1>
- **By transaction reference** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Locked-2>

PUT /transaction/set/unlocked

Summary

Unlocks a transaction. This returns the transaction to the awaiting queue, so that it can be processed by another user, or at a later time.

- **By transaction Id** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Unlocked-1>
- **By transaction reference** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Unlocked-2>

PUT /transaction/set/acknowledged

Summary

Marks the transaction as acknowledged. This is used by API users that process transactions asynchronously. Marking a transaction in this way stops the system from flagging a transaction as delayed.

- **By transaction Id** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Acknowledged-1>
- **By transaction reference** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Acknowledged-2>

PUT /transaction/set/retry

Summary

Marks the transaction as in retry state. This state allows for a temporary alert hold to be placed. You should specify the retry time to wait, before beginning processing again.

- **By transaction Id** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Retry-1>
- **By transaction reference** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Retry-2>

PUT /transaction/setresponse

Summary

Processes your response for the transaction, based on your response code and response message. To use this functionality, you will need to register your response codes in advance with the WorldRemit Operations team, so that they can match the code to the appropriate transaction state above.

- **By transaction Id** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Response-1>
- **By transaction reference** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Response-2>

Transaction Counter

Transactions generated by the WorldRemit system use a unique identifier (GUID) as the primary reference id. For many of our correspondent partners, this can be problematic as they require an incremental numeric identifier for each of our transactions that they process on their system.

To resolve this problem, we have created a set of API functions that provides a unique and incremental numeric counter that can be assigned to a transaction and referenced against our internal identifier.

POST /transactioncounter/create

Summary

This API method creates a transaction counter collection. A transaction counter collection is an incremental sequence of transaction. A transaction can only belong to a collection once, although a transaction can belong to multiple collections.

You can add a description to collection for reference, and can specify the counter start value, which if not supplied defaults to 1.

The full details on how to create a collection can be found at the online portal below:

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Counter-Collection-Create>

POST /transactioncounter/set/{collectionid}/{txnid}

Summary

This method sets a transaction to an existing collection and returns the counter value assigned to the transaction.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Counter-Set>

GET /transactioncounter/read/{collectionid}

Summary

This method reads the current status of a counter collection, including the current counter position.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Counter-Read>

GET /transactioncounter/resolve/{collectionid}/bycounter/{counter}

Summary

This method resolves the Transaction Id from the collection counter id.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Counter-Resolve-1>

GET /transactioncounter/resolve/{collectionid}/bytxnid/{txnid}

Summary

This method resolves the counter id from the Transaction Id.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Counter-Resolve-2>

Transaction External References

For correspondents that generate an external reference against our transaction this can be set during the Transaction State transitions, i.e. Paid, Acknowledged, Lock etc.

The external reference commands can be used to search for your transaction reference against the WorldRemit Id.

For several of our correspondents, their reference id may change during the processing of a transaction. To manage this, the data returned supports multiple reference codes for a single WorldRemit transaction.

POST /transaction/externalreferences/lookup

Summary

This method returns a list of WorldRemit transaction that match the external reference supplied. You can filter the search to specific Correspondent Ids and transaction types. It is

worth noting that our system does not enforce any unique constraint on the external references set against a transaction.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Identifiers-Lookup>

GET /transaction/externalreferences/read

Summary

The method will read the external correspondent transaction references assigned against a WorldRemit transaction. These references can be retrieved by using the WorldRemit transaction id, or our internal references.

- **By transaction Id** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-External-References-Read-1>
- **By transaction reference** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-External-References-Read-2>

Transaction Comment

You can add comment to a transaction, which will be viewed by the WorldRemit Customer Service team.

POST /transaction/addcomment

Summary

Returns the awaiting unprocessed transactions for the API user.

- **By transaction Id** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Comment-1>
- **By transaction reference** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Set-Comment-2>

Transaction Batch

The transaction batch entity is used to group sets on transaction together to allow for group actions to be performed. This is primarily for Correspondents who process transactions through use of a batch file system.

GET /batch/{id}

Summary

Retrieves a batch entity by its id.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Batch-Read>

POST /batch

Summary

Creates a batch entity.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Batch-Create>

POST /batch/complete

Summary

Marks a batch as completed successfully.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Batch-Complete>

POST /batch/cancel

Summary

Marks a batch as cancelled.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Batch-Cancel>

Transaction Batch Lot

The Batch Lot entity is primarily for Correspondents who process transactions through use of a batch file system. The Batch Lot can be used to keep a numeric counter for the transmission files.

POST /batchLot

Summary

Creates a Transaction Batch Lot entity.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/BatchLot-Create>

GET /batchLot

Summary

Reads a Transaction Batch Lot entity by id.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/BatchLot-Read>

PUT /batchLot

Summary

Updates a Transaction Batch Lot entity.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/BatchLot-Update>

Transaction Cancellation

The Transaction Cancellation methods allow you to receive cancellation requests that may be generated for an inflight transaction. This availability of this functionality will depend on whether this capability is configured for your account. You will only receive cancellation requests for transactions that you have previously read from the API.

After you have received a request, you should lock it before doing any further processing to ensure it is not picked up again. You should then set the response marking the cancellation as successful or as failed.

POST /transactioncancelled/awaiting

Summary

Retrieves the awaiting transaction cancellation requests.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Cancellation-Awaiting>

POST /transactioncancelled/lock/{id}

Summary

Locks the Transaction Cancel request. This ensures the request will not be picked up by another API user.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Cancellation-Lock>

POST /transactioncancelled/unlock/{id}

Summary

Unlocks the Transaction Cancel request. This allows the request to be picked up by another API user.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Cancellation-Unlock>

POST /transactioncancelled/setresponse/{id}

Summary

Marks the Cancellation request as completed. You will need to set the success property inside the request body to mark this request as a success or as a failure.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Cancellation-Unlock>

Transaction Change Beneficiary

The Transaction Change Beneficiary requests will be sent when the transaction cannot be processed due to incorrect recipient details, or when a sender has requested a change of recipient. You will only be sent these requests for transactions that you have previously processed.

POST /transactionchangebeneficiary/awaiting

Summary

Retrieves the awaiting change beneficiary requests.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Change-Beneficiary-Awaiting>

POST /transactionchangebeneficiary/lock/{id}

Summary

Locks the Change Beneficiary message so that it will not be picked up by another API user.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Cancellation-Lock>

POST /transactionchangebeneficiary/unlock/{id}

Summary

Unlocks the Change Beneficiary message so that it can be picked up by another API user.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Change-Beneficiary-Unlock>

POST /transactionchangebeneficiary/setresponse/{id}

Summary

Marks the Change Beneficiary message as completed. You will need to set the success property inside the request body to mark this request as a success or as a failure.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Change-Beneficiary-Set-Response>

Transaction Status

Transaction Status messages will be generated in two circumstances:

1. The original transaction generated an error during processing, and we are unsure of the actual status of a transaction. These types of requests are generated manually. They can be generated by a customer calling the WorldRemit customer service team, or by our Operations team, for a transaction that has failed and is in an indeterminate state.
2. The transaction is currently in an Awaiting status, and we are waiting to find the final status of the transaction. This will happen for transactions that are processed asynchronously. If you wish to remove a transaction from this queue for a period of time, for example if your remote gateway is down, you should set the transaction to '[Retry](#)' and specify a time before it should be returned to the Awaiting state.

POST /transactionstatus/awaiting

Summary

Retrieves the waiting transaction status requests, with the type 1 message above given priority over the type 2 async status requests.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Status-Awaiting>

POST /transactionstatus/lock/{id}

Summary

Marks the transaction status request as locked so that it will be removed from the Awaiting queue and cannot be retrieved again.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Status-Lock>

Transaction Notepad

The Transaction Notepad allows you to store additional data for a transaction on the WorldRemit system. This functionality is typically used to stored third party reference for a transaction while it is being processed by a downstream system. The Notepad consists of a set of key/value pairs that can be stored against the Transaction Id.

POST /transactionnotepad

Summary

Creates a Transaction Notepad entity.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Notepad-Create>

GET /transactionnotepad

Summary

Retrieves a Transaction Notepad by Transaction id or by Transaction reference.

- **By transaction Id** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Notepad-Read-1>
- **By transaction reference** - <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Notepad-Read-2>

PUT /transactionnotepad

Summary

Updates a Transaction Notepad entity.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Transaction-Notepad-Update>

Validation

Validation is used to verify a recipient's details before the transaction process begins. We would prefer that all Correspondents support this new functionality, as this significantly reduces the number of failed transactions, and the number of beneficiary/recipient changes that you will have to process.

These requests are time critical; they should be processed within seconds, as the WorldRemit customer will be trying to validate the recipient's details in real time using the web site or our mobile application.

If you are unable to process these requests within a short time, please tell the WorldRemit Operations team, and this feature will be disabled for your account.

You should also set up an outgoing Webhook for these Validation request, so that you are notified immediately of an awaiting request.

POST /validation/awaiting

Summary

This API method retrieves the list of awaiting Validation requests.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Validation-Awaiting>

PUT /validation/complete

Summary

Processes the response message to complete a validation request.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Validation-Complete>

Webhooks

A Webhook is a registered URL endpoint that you define, that our servers will call when the specified action occurs on our system.

Webhooks allow for instant notification of awaiting requests for your account, and in many cases, removes the need for you to constantly poll our API for new requests.

The supported Webhook poll actions are:

- **Heartbeat** – this is timer poll from our system which ensure that the connection is open and active to your servers.
- **Validate Recipient Request** – this notification will be sent when there are pending validation requests waiting to check the beneficiary/recipient details.
- **Transaction Request** – this notification is triggered when a new transaction arrives for processing.
- **Cancellation request** – this notification is triggered when a transaction cancellation request is raised.
- **Change Beneficiary Request** – this notification is triggered when a customer wishes to change the beneficiary/recipient details for an existing transaction.

- **Transaction Status Request** – this notification will be triggered when we require a status update for a transaction that you are processing on your system. We typically use this when a transaction is processed asynchronously, or when a transmission error occurs during processing.

The endpoint that you register will be called with the URL that you specify. You can register up to 32 specific endpoints with your account. We recommend that you use the **https** protocol for your endpoints, but we will support **http**. Please note: no sensitive information (i.e. transaction body or transaction id) will be sent along with the Webhook call-back.

For the endpoint host, you can use either a DNS resolved name or a public IPv4 address. Private IPv4 address, and IPv6 addresses, are not supported.

If you supply a **https** secured remote endpoint, you can also specify that we sign the outgoing request, using our mutual-SSL certificate.

Appending the Webhook action type to the URL

You can set **EndpointAppendMode** property to define whether you want the notification type to be appended to your URL. The default setting is to append the numeric value of the Notification type to the end of the URL. For a **TransactionStart** notification, this will mean that the calling Endpoint will be appended with the value `'/2'`.

If your registered Webhook Endpoint is:

- `'https://mybank.com/callback'`

If the value is set to **AppendToUrl** then the actual URL called will be:

- `'https://mybank.com/callback/2'`

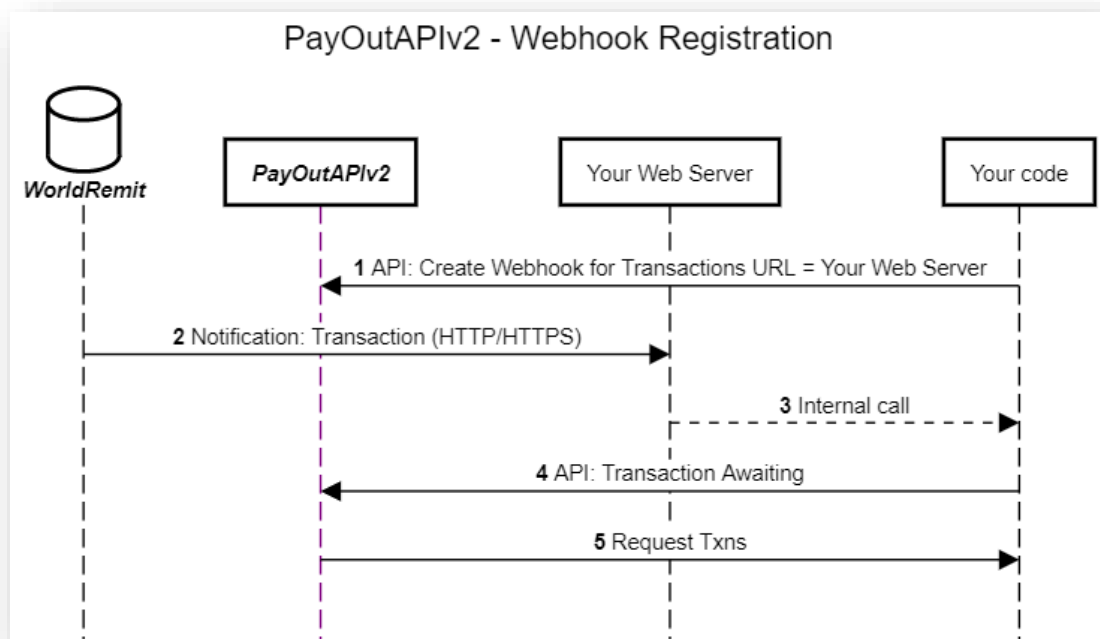
If however, you set to **AppendAsQuery** the URL call will be:

- `'https://mybank.com/callback?worldremitaction=2'`

The call-back will be a HTTP GET request to the URL that you specify. Http redirects will not be allowed. If you specify a https endpoint, we will check for a valid TLS certificate and confirm that it has not been revoked. If your TLS certificate is not valid, the call will be aborted. Your web server will need to support TLS 1.2 or higher.

Webhook initial registration example

The diagram above shows how a Webhook can be used. You first register the Webhook and inform



the WorldRemit systems which event types you wish to receive notifications for. Then when an event occurs that matches your registration, such as a new transaction being ready to process, the system will call your URL endpoint to inform you.

You will then call the standard 'Awaiting' methods for the type of event required: Transaction, Cancellation, Change Beneficiary, Transaction Status, etc. Additional example of an End-to-End transaction flow can be found in the API Examples section at the end of the document.

POST /webhook

Summary

Creates the specified webhooks for your account.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Webhook-Create>

GET /webhook/{id}

Summary

Returns the specific registered webhooks for your account.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Webhook-Read>

GET /webhook

Summary

Returns all the registered webhooks for your account.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Webhook-Read-All>

DELETE /webhook/{id}

Summary

Deletes the specific webhook, which is registered for your account.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Webhook-Delete>

DELETE /webhook

Summary

Deletes the all the registered webhooks on your account

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Webhook-Delete-All>

User Capabilities

This API call allows you to determine your specific User Id, and the specific Correspondent Ids assigned to you, which you can use to retrieve transactions for a specific product type. The data returned contains a description of the transaction grouping, and the supported products for that grouping.

GET /capabilities

This method is used to read the capabilities for the credentials that we provide to you.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/User-Capabilities-Read>

User Notepad

This section is a Notepad entity that allows you to maintain specific settings for your account

The Correspondent Notepad entity can be used to store a set of key-value pairs against each of your assigned transaction payment groups.

POST /usernotepad

Summary

Creates a new Notepad entity for your account:

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/User-Notepad-Create>

GET /usernotepad

Summary

Retrieves your Notepad entity for your account:

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/User-Notepad-Read>

PUT /usernotepad

Summary

Updates your Notepad entity for your account:

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/User-Notepad-Update>

Correspondent Notepad

The Correspondent Notepad entity can be used to store a set of key-value pairs against each of your assigned transaction payment groups. These groups and their unique identifiers can be determined by calling the User Capabilities method.

POST /correspondentnotepad

Summary

Creates a new Correspondent Notepad entity:

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Correspondent-Notepad-Create>

GET /correspondentnotepad/{id}

Summary

Retrieves a Correspondent Notepad by Correspondent Id:

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Correspondent-Notepad-Read>

PUT /correspondentnotepad

Summary

Updates a Correspondent Notepad entity:

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/payout/operations/Correspondent-Notepad-Update>

SANDBOX API OVERVIEW

This API is only available on our test environments. With this API you can create test transactions, and subsequent cancellation requests, or a change of beneficiary for your test transaction, using our API processing system.

It is important to note that the Sandbox API is not available in our SIT, UAT or Production systems; it is only provided for our test Sandbox environment.

This API is designed to help speed up your integration work with our systems. You can use it to write automated integration testing. The API will always be available for your use for your testing and will always match our current production environment version.

Paths

This section contains the Sandbox API method calls. These calls are grouped around common functionality below.

In addition, each of the API method call documented contain the direct link to the API portal definition, which has the complete definition. You will need to have registered with the portal to view these links (see **Registering with the Developer Portal** for details).

Transaction Create

We have added five new shortcut methods in our Sandbox API that allow you to quickly create simple test transactions with the necessary parameters pre-filled for the type of product.

Each call will return the full transaction object that has been generated. The transaction that is returned is assigned a unique identifier in the form of a [GUID](#) in the Id parameter. This id should be used for further reference on the Sandbox APIs, such as raising a cancellation request, or a change beneficiary request.

Please note that the **StatusText** property will be set to:

"WARNING: THIS IS A TEST TRANSACTION - DO NOT PROCESS FOR PAYMENT".

This should be checked on your system to ensure that you do not mix production and testing gateways when requesting transactions for processing.

POST /v2/sandbox/transactioncreate/cash

Summary

This method is used to create a test cash transaction.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/sandbox/operations/CreateTransactionCash>

POST /v2/sandbox/transactioncreate/cash2FA

Summary

This method is used to create a test cash transaction with a mobile phone number specified in the request body. This will allow you to test receiving the verification code as a text message.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/sandbox/operations/CreateTransactionCash2FA>

POST /v2/sandbox/transactioncreate/bank

Summary

This method is used to create a test bank-transfer transaction.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/sandbox/operations/CreateTransactionBank>

POST /v2/sandbox/transactioncreate/mobilemoney

Summary

This method is used to create a test mobile money transaction.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/sandbox/operations/CreateTransactionMobileMoney>

POST /v2/sandbox/transactioncreate/sendtocard

Summary

This method is used to create a test card transfer transaction.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/sandbox/operations/CreateTransactionSendToCard>

POST /v2/sandbox/transactioncreate/airtimetopup

Summary

This method is used to create a test airtime top-up transaction.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/sandbox/operations/CreateTransactionAirtimeTopup>

Transaction

We have provided additional methods in our Sandbox environment to allow you to send test transactions through this system.

Each transaction that is processed is assigned a unique identifier in the form of a [GUID](#). When you have submitted a transaction for processing, this identifier will be returned to you in the transaction response body and should be used for further reference on the Sandbox APIs.

POST /v2/sandbox/transaction/ingest

Summary

This method is used to ingest a test transaction.

Please note that the **StatusText** property will be set to:

"WARNING: THIS IS A TEST TRANSACTION - DO NOT PROCESS FOR PAYMENT".

This should be checked on your system to ensure that you do not mix production and testing gateways when requesting transactions for processing.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/sandbox/operations/IngestTransaction>

GET /sandbox/transaction/read/{id}

Summary

This method can be used to read the full transaction. When reading transactions through the PayOut API, certain sensitive sender and recipient fields will be removed, due to legal compliance reasons. This "Transaction Mask" will be configured by our Operations team, based on the agreement terms. This API call allows you to read the full original transaction fields for debug purposes.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/sandbox/operations/Read>

GET /sandbox/transaction/read[?wrid][&reference]

Summary

This method allows you to read the unobscured transaction using the WorldRemit Id or Reference.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/sandbox/operations/Read-By-Reference>

Transaction Cancel

You can mark an existing transaction for cancellation.

POST /v2/sandbox/transactioncancel/ingest

Summary

This method marks an existing transaction for cancellation.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/sandbox/operations/IngestCancellation>

Transaction Beneficiary Change

You can mark an existing transaction for a change beneficiary request. Whether this is processed will depend on the current state of the transaction.

POST /v2/sandbox/transactionchangebeneficiary/ingest

Summary

This method allows you to upload a new Transaction with a change of recipient.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/sandbox/operations/IngestChangeBeneficiary>

Transaction State

During development and debugging, it is sometimes necessary to diagnose the state of a transaction. This section allows you to call the API and find the transaction processing history.

For a more detailed of the transaction state see this [section](#).

GET /sandbox/transactionstate/history/{id}

Summary

This method allows you to read the state transition history of a transaction. In addition, you will be able to read additional processing data, stored against your transaction. Specifically, for Correspondents that use two-factor authentication codes for cash pickup which, you can read the code that has been generated, which will allow you to complete your integration tests.

- <https://corint-apim-tst-wr.portal.azure-api.net/docs/services/sandbox/operations/History>

API EXAMPLES

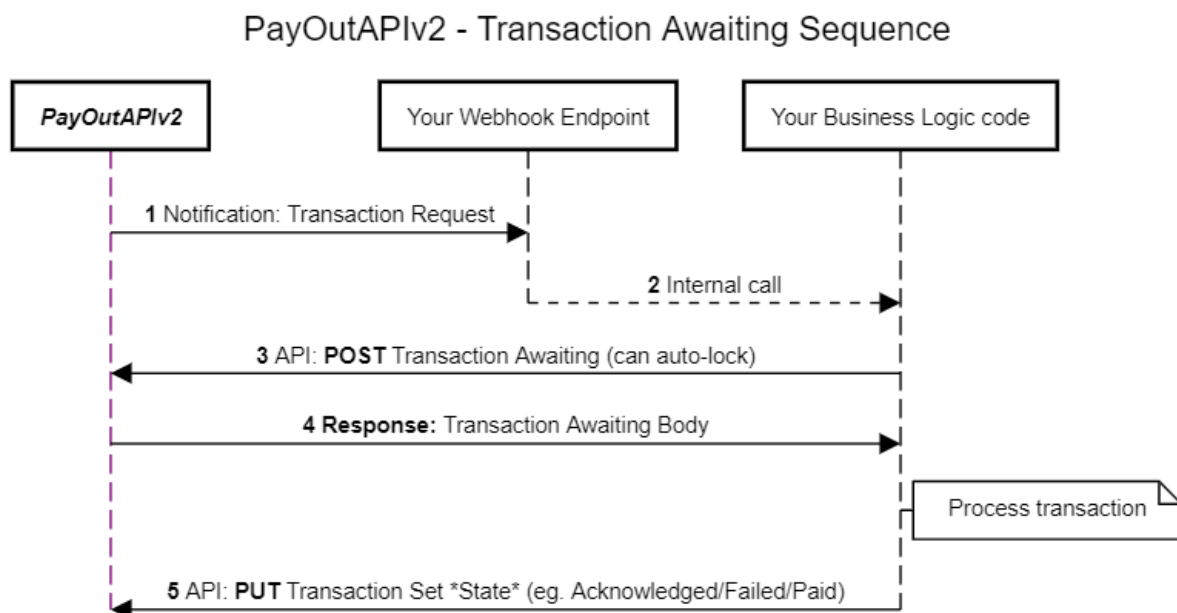
This section outlines specific examples of how you can use our PayOut API to process transactions. We have provided code examples in C# using our NuGet library.

Getting started

Before starting development against the API, it is useful to check what capabilities have been set up for your account. If these are not what you are expecting, then please contact the WorldRemit operations team.

The below examples illustrate common use cases. An integration may have a combination of these flows depending on the Product Type supported and any additional business rules. Therefore, this is not an exhaustive list of use cases.

Transaction Awaiting processing using Webhooks



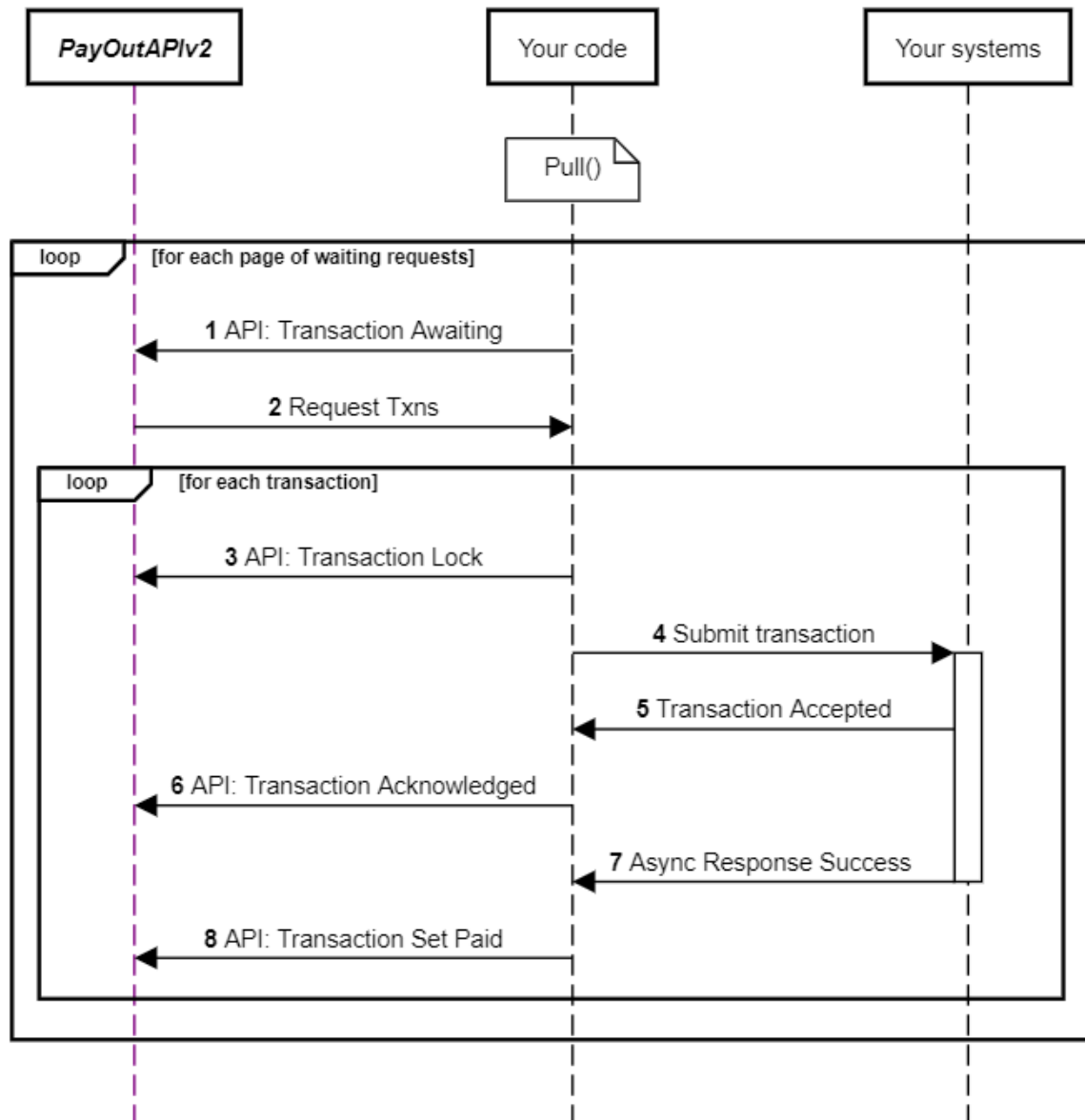
Cash Pick-up with SMS verification code (S2FA)

The new PayOut API, can send a second SMS to the recipient's mobile device which includes a 7-digit number to be presented when picking up cash. This is enabled in jurisdictions where this is a legal requirement and will be enabled by our Operations team when your account is configured.

For the 7-digit verification code, the last digit of this code is a [Luhn](#) checksum digit, which can optionally be used to ensure that the code has been typed correctly in your application.

Asynchronous flow

PayOutAPIv2 - Async flow



[Document Ends]